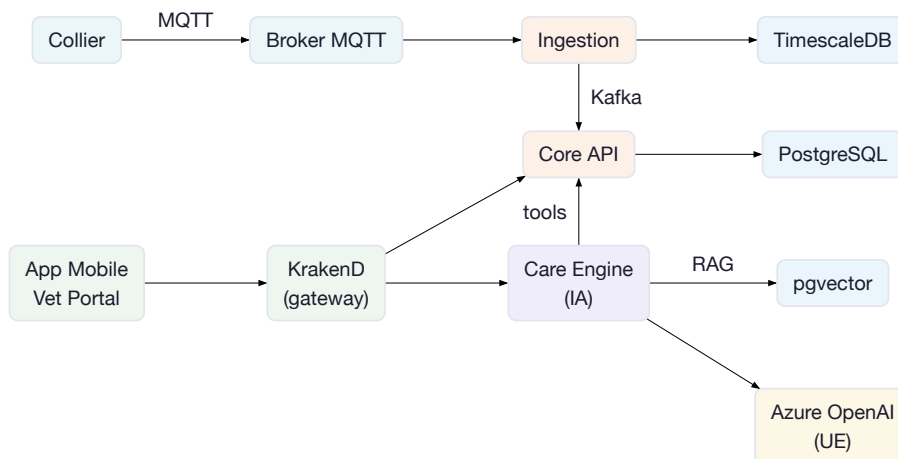


1 Architecture technique

1.1 Vue d'ensemble (simplifiée)

Le système s'organise en trois zones : clients et accès, services métier (Rust) autour d'un bus d'événements, et données. L'IA (Care Engine) est isolée et bornée. Schéma des flux principaux :



Version interactive complète (toutes les briques, flux et parcours) sur pawrise-care.com.

1.2 Choix techniques justifiés

Chaque décision structurante : besoin, alternatives écartées, décision, et trade-off assumé.

Choix	Alternatives écartées	Décision & trade-off
Rust (services métier)	Node, Python, Go, C#	Perf + sûreté mémoire, empreinte minimale. Coût dev ~2,5× assumé (mitigé : monolithe-first, Python pour l'IA, réintroduction ciblée).
MQTT / EMQX (entrée IoT)	HTTP, CoAP, passerelle maison	Standard IoT : pub/sub léger, QoS, offline gratuits. Un composant de plus à opérer.
Apache Kafka (bus)	RabbitMQ, appels directs	Débit, rétention, découplage. Plus gourmand en RAM (dimensionne le cluster).
PostgreSQL + TimescaleDB Mongo + Influx + Qdrant séparés + pgvector		Une seule famille : transactionnel, séries temporelles, vectoriel. Bascule Qdrant possible.
Pipeline LangGraph borné	Agent LLM autonome	Chaque étape testable, responsabilité juridique portée par le pipeline (ADR-001).
Azure OpenAI EU + cascade + Cohere	LLM hors UE, modèle unique	Data residency UE, ~0,03-0,05 €/conv. Dépendance Microsoft mitigée (interface LLMPProvider).
Kubernetes auto-géré + GitOps	PaaS, managé d'emblée	Économique et souverain ; plus d'ops, mitigé par l'laC. Managé EU au scale.
OIDC maison	Auth0, Keycloak, Cognito	Souveraineté + défi formateur ; coûteux (~4-6 mois-homme), revue de sécurité obligatoire.